

Speaker Notes — CIBB 2026 Talk

GPU-accelerated single-cell and spatial transcriptomics on NVIDIA DGX H100

Target length ~12 minutes · 14 slides · ~50 s/slide. Full spoken script below, one block per slide.

Slide 1: The scale problem

Good morning. Single-cell RNA sequencing has become the workhorse for studying cellular heterogeneity, and datasets have exploded in size — a million cells is now routine, and spatial platforms like Visium HD push us into hundreds of thousands of measurement bins per tissue section.

The problem is that the dominant analysis tools — Scanpy for single-cell, Squidpy for spatial — run on CPU. At atlas scale, a complete pipeline can take many hours, sometimes a full day. So the practical question we set out to answer is simple: can GPUs make this kind of analysis routine — and, crucially, can they do it without changing the biological conclusions we draw? That last part matters, because a fast wrong answer is worthless. Let me show you what we found.

(~128 words, ~51 s at 150 wpm)

Slide 2: What we benchmarked

Here's the setup. For single-cell, we ran on a DGX H100 node — eight H100 GPUs, each with 80 gigabytes of memory, backed by 112 CPU cores and 2 terabytes of RAM. Spatial benchmarks ran on a more modest local workstation with a single consumer RTX 4090, to show the results aren't limited to elite hardware.

On the software side we compared Scanpy on CPU, using all 100 threads, against rapid-singlecell on GPU — that's a near drop-in replacement built on the RAPIDS ecosystem. Everything ran inside Singularity containers built from a fixed NVIDIA base image, so the whole thing is reproducible.

We asked five questions, and they organise the rest of the talk: how big is the speedup, both per-step and end-to-end; how well does it scale across multiple GPUs; do CPU and GPU give the same biology; how many cells can we push through a single node; and does any of this carry over to spatial transcriptomics. Let's take them in turn.

(~164 words, ~66 s at 150 wpm)

Slide 3: The pipeline is identical on both sides

A benchmark is only fair if both pipelines do exactly the same work, so we were strict about that. Same input — the 10x Genomics 1.3-million mouse brain dataset, uniformly subsampled to five sizes from ten thousand up to the full 1.3 million. Same parameters at every step: quality control, library-size normalisation, two thousand highly variable genes, fifty principal components, a k-nearest-neighbour graph with k of fifteen, Leiden clustering, UMAP, and Wilcoxon differential expression. Same random seed, and five independent repeats of every configuration so we have error bars, not single runs.

And rather than just assuming the outputs match, we measured concordance directly at each stage — gene-selection overlap, PCA correlation, clustering agreement, and differential-expression correlation. Those numbers come back later in the talk.

(~125 words, ~50 s at 150 wpm)

Slide 4: Up to 120× faster end-to-end

This is the headline. At the full 1.3 million cells, the eight-GPU pipeline finished the entire analysis in 435 seconds — just over seven minutes — versus 52,000 seconds on CPU, which is fourteen and a half hours. That's a 120-fold speedup. A analysis you'd start and leave overnight now finishes before your coffee is cold.

But the interesting story is in this heatmap, which breaks it down step by step. The speedups are wildly uneven — they span two orders of magnitude. Normalisation, which is trivially parallel element-wise arithmetic, hits 329-fold. Neighbour-graph construction and UMAP are in the tens-of-times range. And notice the one place where CPU actually wins: data loading, at the far left, is slightly faster on CPU because the GPU pays a fixed cost to initialise its CUDA context. So the GPU isn't uniformly faster — it's dramatically faster where the work is parallel, and the profile of your pipeline determines what you actually get.

(~159 words, ~64 s at 150 wpm)

Slide 5: Where the time goes

Here's the same result seen as absolute wall time rather than ratios, and it makes the bottleneck obvious. On the CPU side — the tall bars — two steps eat almost all the time: differential expression testing and neighbour-graph construction. On the GPU side, everything collapses down to seconds; you can barely see the individual steps.

This is why the per-step view matters. If you only accelerate the cheap steps, you save nothing. The value of the GPU here is that it crushes exactly the two operations that dominated the CPU runtime. That's the difference between a marketing speedup and a useful one.

(~103 words, ~41 s at 150 wpm)

Slide 6: More GPUs help less than you'd hope

Now a result that surprised us, and that has real practical consequences. You might expect that if two GPUs are good, eight are four times better. They're not. Look at these curves — beyond two GPUs they're almost flat. At 1.3 million cells, eight GPUs were only twelve percent faster than two.

Why? Because when we profiled an eight-GPU run, seventy-five percent of the wall time was CPU-side preprocessing — loading, filtering, normalising — which doesn't touch the GPUs at all. Only four percent of the time was in the part that actually benefits from more GPUs, the distributed PCA and neighbours. This is textbook Amdahl's law: your speedup is capped by the part you didn't parallelise. The preprocessing is the anchor.

(~122 words, ~49 s at 150 wpm)

Slide 7: So use the node differently

(no notes)

(~2 words, ~1 s at 150 wpm)

Slide 8: Practical takeaway

The scaling result flips into a useful piece of advice. If adding GPUs to one job barely helps, then the efficient way to use an eight-GPU node is not to throw all eight at a single analysis — it's to run eight separate analyses, one per GPU, in parallel. You get eight times the throughput instead of twelve percent off one job.

And the deeper point is about accessibility. Because nearly all the benefit lives on a single GPU, you don't need a half-million-dollar DGX to get it. A single H100 rented in the cloud for two to four dollars an hour gives you almost the entire speedup. This democratises atlas-scale analysis — it's not gated behind owning big iron.

(~120 words, ~48 s at 150 wpm)

Slide 9: The biology is preserved

This is the slide that makes the speed meaningful. A faster pipeline is only useful if it gives you the same answer, so we compared the outputs directly.

The gene selection was identical — a Jaccard index of exactly one, the same two thousand genes. The PCA loadings correlated perfectly. Leiden clustering agreement was between 0.91 and 0.96 on the adjusted Rand index, with normalised mutual information above 0.95. And the differential-expression fold-changes correlated at 0.95. The small residual differences aren't errors — they come from GPU floating-point non-determinism and from stochastic steps like Leiden and UMAP, and they're well within the range you'd see just re-running the CPU pipeline with a different seed. So the message is clean: the GPU gives you the same biology, an order of magnitude or two faster.

(~133 words, ~53 s at 150 wpm)

Slide 10: How far can one node go?

Next we pushed a memory-optimised version of the pipeline to find the breaking point on a single node. It succeeded at 11.9 million cells — 119 minutes — and failed at twelve million, when Leiden clustering ran out of memory.

But here's the counterintuitive part, shown in this memory profile. The thing that ran out was not GPU memory. Aggregate VRAM across all eight GPUs stayed flat and low — only 7.6 percent of the 640 gigabytes available. The binding constraint was CPU RAM, which peaked at 535 gigabytes during preprocessing. So the message for anyone planning a large run: to go bigger, you need more system RAM, not more or bigger GPUs. That reframes how you'd spec a machine for this work.

(~123 words, ~49 s at 150 wpm)

Slide 11: Differential expression: pseudo-bulk wins twice

Differential expression deserves its own slide, because it was the single biggest time sink on CPU. We benchmarked three approaches at 3.4 million cells. The cell-level t-test on CPU took over 5,500 seconds. GPU Wilcoxon brought that to 826. But pseudo-bulk aggregation — where you sum counts within each cluster-donor group before testing — finished in 128 seconds, 44 times faster than the CPU t-test.

And the beautiful thing is that pseudo-bulk isn't a speed-versus-correctness tradeoff. It's also the more statistically rigorous choice, because it avoids pseudoreplication — treating thousands of cells from one donor as independent samples, which inflates significance. So for multi-donor experiments, the fastest method is also the most correct one. You rarely get to say that.

(~120 words, ~48 s at 150 wpm)

Slide 12: It generalises to spatial transcriptomics

Finally, does this carry beyond single-cell? We ran the same kind of benchmark on spatial transcriptomics — three 10x Visium platforms — and on the modest RTX 4090, not the DGX.

End-to-end we saw up to 51-fold speedup on the high-resolution Visium HD data. And one operation stood out: co-occurrence analysis, which is an order-n-squared computation on CPU, sped up by a factor of 3,272 when parallelised across GPU threads. That's the kind of quadratic operation GPUs were made for. Concordance was again essentially perfect — the spatial autocorrelation statistics, Moran's I and Geary's C, correlated at 0.9995 or better. One honest caveat: clustering agreement diverged on the very highest-resolution data, because the CPU and GPU Leiden implementations break near-degenerate partitions differently — that's a known algorithmic difference, and it's in the paper.

(~133 words, ~53 s at 150 wpm)

Slide 13: Takeaways

Let me pull it together into five things to remember. One: GPU acceleration gives up to a 120-fold end-to-end speedup with biologically concordant results — same answer, dramatically faster. Two: the bottleneck is CPU-side preprocessing, not GPU compute, which is why most of the value is on a single GPU and why multi-GPU scaling is disappointing. Three: the ceiling on how big you can go is CPU RAM, not GPU memory — we hit 11.9 million cells on one node. Four: for differential expression, pseudo-bulk is both the fastest and the most statistically correct choice. And five: it all generalises to spatial transcriptomics, over 50-fold, even on a consumer graphics card.

Everything — the code, the Singularity containers, and every benchmark JSON — is public on GitHub. Thank you, and I'm happy to take questions.

(~135 words, ~54 s at 150 wpm)

Slide 14: Thank you

Thank-you slide with contact details and funding acknowledgements — the UPSCALE and CONVECS HPC infrastructure at Padova that provided the DGX H100, and our funding from the national DARE plan and the BIRD START programs. Leave this up during questions.

(~40 words, ≈ 16 s at 150 wpm)

Total script: ~1607 words ≈ 10.7 min at 150 wpm (≈ 12.4 min at a measured 130 wpm).